Amazon Product Recommendations: A comparative study of various algorithms

Soham Bhagwat[1†] and Srikar Kashyap Pulipaka[1†]

[1*]Luddy School of Informatics, Indiana University, Bloomington.

Contributing authors: sobhagwa@iu.edu; spulipa@iu.edu;
[†]These authors contributed equally to this work.

Abstract

Recommendations play an increasingly important role for consumer preference in e-commerce websites. In this paper, we analyze the results of various traditional and graph machine learning based algorithms on a popular dataset to benchmark their performance. We find that the graph machine learning based models perform better than traditional methods. We discuss our findings and propose possible future directions for the project.

Keywords: Graph Analytics, Recommendations, Machine Learning

1 Introduction

Recommendation systems are an integral part of the internet and e-commerce platforms. They influence our choices on what products to buy, what movies to watch and whom to add as friends on social media. These systems often provide recommendations based on the past interactions of a user by comparing it with interactions of similar users across the platform. They are important for improving the user experience and generating business for the organizations.

Collaborative filtering is the most widely used and traditional method for generating recommendations. It works on the principle of finding similar users based on their past behavior. It's limitations include the sparcity in data due to lack of data for interactions for all the possible items.

On the other hand, techniques based on Graph networks and graph neural networks have become popular in recent years, owing to their capability to leverage and model the underlying graph structure of the relationships. Some popular techniques in this category include GraphSAGE and Neural Graph Collaborative Filtering (NGCF) both of which combine the power of neural networks with the graph structure to provide state of the art performance. [1]

In this analysis, we check the performance of three recommendation techniques: traditional collaborative filtering, GraphSAGE and NGCF. We evaluate their performance on a common dataset and calculate Hit Rate (percent of accurate recommendations) for all the techniques. We also discuss their performance parameters, their strengths and weaknesses. The Methods section provides an overview of all the three sections and their theoretical underpinnings. The Experimental Setup section describes how the experiments were conducted and the description of dataset. The Results section discusses the obtained results and their implications, while the Conclusions section discusses key takeaways and future scope of the project.

2 Methods

We predominantly survey three methods for recommender system.
1. A traditional nearest neighbor based collaborative filtering.
2. GraphSAGE based approach.

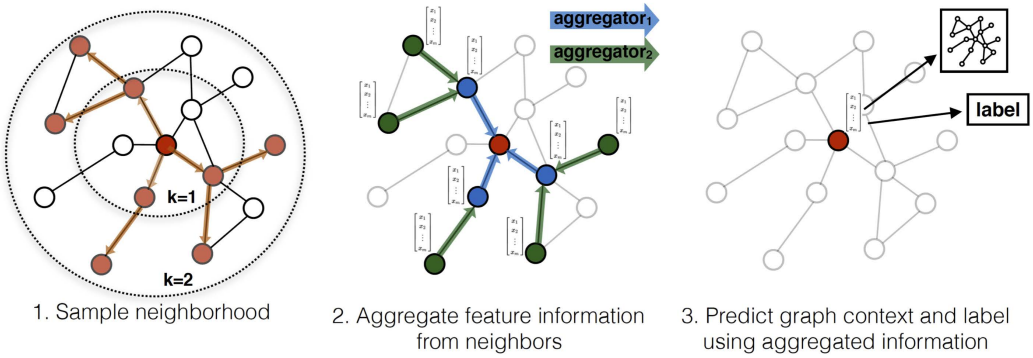

Fig. 1 Illustration of how GraphSAGE works. Figure from the original paper.[2]

3. Neural Graph Collaborative Filtering (NGCF) approach

2.1 User-Item Based Collaborative Filtering

Collaborative methods for recommender systems are methods that are based solely on the past interactions recorded between users and items in order to produce new recommendations and are stored in the so-called "user-item interactions matrix".
If we assume n users and d items,dimensionality of user–item rating matrix X is n × d.
Rows and columns in X correspond to users and items profile, respectively.The User-Item collaborative filtering reccomends items by finding similar users to the current user for whom we are trying to make a reccomendation.

$$\text{similarity}(u, v) = \cos(\theta) = \frac{\sum_{i \in I_{uv}} R_{ui} \cdot R_{vi}}{\sqrt{\sum_{i \in I_u} R_{ui}^2} \cdot \sqrt{\sum_{i \in I_v} R_{vi}^2}} \tag{1}$$

In this equation, similarity(u, v) represents the cosine similarity between users u and v. The angle θ between the two users is calculated using the cosine function, which takes the dot product of their rating vectors and divides it by the product of their vector magnitudes. This above given formula is used in user-item collaborative filtering to compute the similarity between users based on their rating patterns.

Algorithm 1 User-Item Collaborative Filtering Algorithm

```
Create a user-item matrix R, where rows represent users and columns represent items. The values in
the matrix represent ratings (or interaction) between users and items.
Compute the similarity between users by comparing their rating patterns. This can be done using
methods such as cosine similarity or Pearson correlation.
for each user do
    Find the most similar users (neighbors) in the training set.
    for each item i do
        Predict the rating that user u would give to item i by taking a weighted average of the ratings
given by the nearest neighbors of user u who have rated item i.
    end for
end for
Evaluate the hit rate of the algorithm
```

2.2 GraphSAGE

GraphSAGE is an unsupervised method used to generate low-level embeddings for nodes in a graph. These representations can then be used for down-stream tasks like link prediction, node classification and node clustering. The primary advantage of GraphSAGE lies in the fact that it doesn't need the entire graph to generate embeddings; it uses local neighbourhood using sampling and aggregation. It is especially useful for leveraging node feature information

The following algorithm briefly illustrates the various steps involved in GraphSAGE. [2]

1. Take the nodes and features and construct a graph G. We also define a fixed number of neighbours to use for sampling at this stage.
2. We do neighbourhood sampling and aggregate the feature vectors by using a neural network. We capture the local neighbourhood information in this step for each node.
3. We pass this aggregation along with the feature vector of the node (by concatenating them) to obtain an embedding for the node.
4. We use this embedding for any task following it like link prediction, classification etc. We define a loss function to measure the error at this stage.
5. Repeat these steps for a fixed number of iterations to get the final embeddings for the nodes.

We can use various methods of aggregation including mean/average aggregation and max aggregation. We use mean aggregation for our analysis.

2.3 Neural Graph Collaborative Filtering

The third model we experimented was Neural Graph Collaborative Filtering. This model uses learnable collaborative filtering methods in the form embeddings which are vectorized form of user - item interactions similar to that of traditional CF.But this traditional method may lack in capturing collaborative signal,which is latent in user-item interactions to reveal the behavioral similarity between users (or items).

The NGCF uses high order connectivity (Fig.1) from user-item interactions to encode collaborative signal in the interaction graph structure.The left subfigure of the user-item interaction graph highlights the user of interest, u1, labeled with a double circle. The right subfigure illustrates the tree structure that expands from u1, where the high-order connectivity represents the path that extends from any node with a path length greater than 1 to u1. This high-order connectivity includes a vast amount of meaningful semantics that carry collaborative signals[3].
Furthermore, there are usually three important steps involved in the NGCF framework. We initialise the user-item embedding.Then, the higher-order connectivities are taken in mind and the embedding are refined throufh multiple propagation methods.The final layer of the model combines the enhanced embeddings from various propagation layers and generates the affinity score for a given user-item pair

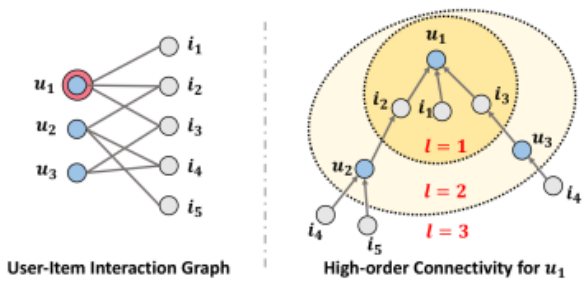

Fig. 2 An illustration of the user-item interaction graph and the high-order connectivity. The node u1 is the target user to provide recommendations for.Image used as is from the original paper[3] .

The following steps are the brief overview of the methodologies involved in the NGCF framework:

1. User and items are nodes and the edges between them represent interaction.The User-Item iteractions are represented as a bipartite graph.(Fig.1)
2. The embedding vectors are arranged in a parameter matrix E.The user and items are mapped through an embedding layer into a low-dimensional vector in the latent space.

$$E = [\mathbf{e}_{u_1}, \ldots, \mathbf{e}_{u_N}, \mathbf{e}_{i_1}, \ldots, \mathbf{e}_{i_M}], \tag{2}$$

   where $\mathbf{e}_{u_i}$ and $\mathbf{e}_{i_j}$ are the embedding vectors.
3. The information between the nodes in the graph is propagated using a graph convolutional layer. Each node's embedding is updated by aggregating the embeddings of its neighbors using a weighted sum. Specifically, the message construction equation is defined as:

$$\mathbf{m}_{u \leftarrow i} = \frac{1}{\sqrt{|\mathcal{N}_u| \cdot |\mathcal{N}_i|}} (\mathbf{W_i} e_i + \mathbf{W_2}(e_i \odot e_u)) \tag{3}$$

where $\mathbf{m}_{u\leftarrow i}$ is the message sent from item i to user u(i.e the message propagation). $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d' \times d}$ weight matrices that are trained are used to extract valuable information for propagation, and the transformation size is denoted by d'.Here $(e_i \odot e_u)$ simply means element-wise product or more succintly passing more messages from the similar items.

The message aggregation equation is then used to combine the messages from all neighbors of a node, using the equation:

$$\mathbf{e}_u^{(l)} = LeakyReLu\left(\mathbf{m}_{u\leftarrow i}^{(l)} + \sum_{i\in\mathcal{N}(i)} \mathbf{m}_{u\leftarrow i}^{(l)}\right) \tag{4}$$

4. On similar lines multiple graph convolutional layers can be stacked to learn higher-order connectivity patterns in the graph.By stacking multiple embedding propagation layers, the collaborative signal is smoothly integrated into the process of learning representations.
5. The final prediction score for a user-item pair is computed as the dot product of their respective embedding vectors in the latent space.

$$\hat{y}_{NGCF}(u,i) = e_u^{*^T} e_i^*$$

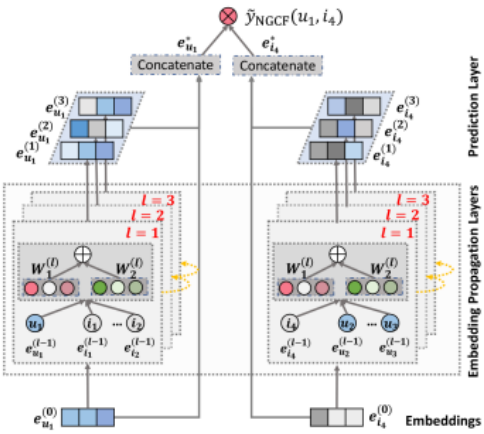

Fig. 3 Figure 2: An illustration of NGCF model architecture (the arrowed lines present the flow of information). The representations of useru1 (left) and itemi4 (right) are refined with multiple embedding propagation layers, whose outputs are concatenated to make the final prediction.Image used as is from the original paper[3]

3 Experimental Setup

3.1 Dataset

The dataset used in this project is the Amazon Review and Metadata dataset by Jianmo Ni et.al. It consists of around 233.1 million reviews for over 15 million products along with their metadata like product description, category information, etc. [4]

The product review section of the dataset consists of the following attributes.

- Image of Review (if available)
- Overall Rating
- Number of helpful votes
- Is a verified user
- Review Time and reviewer ID
- Product ASIN
- Reviewer Name
- Reviewer Text and Summary

The Metadata section of the dataset consists of the following attributes for each product.

- Product Description
- Category

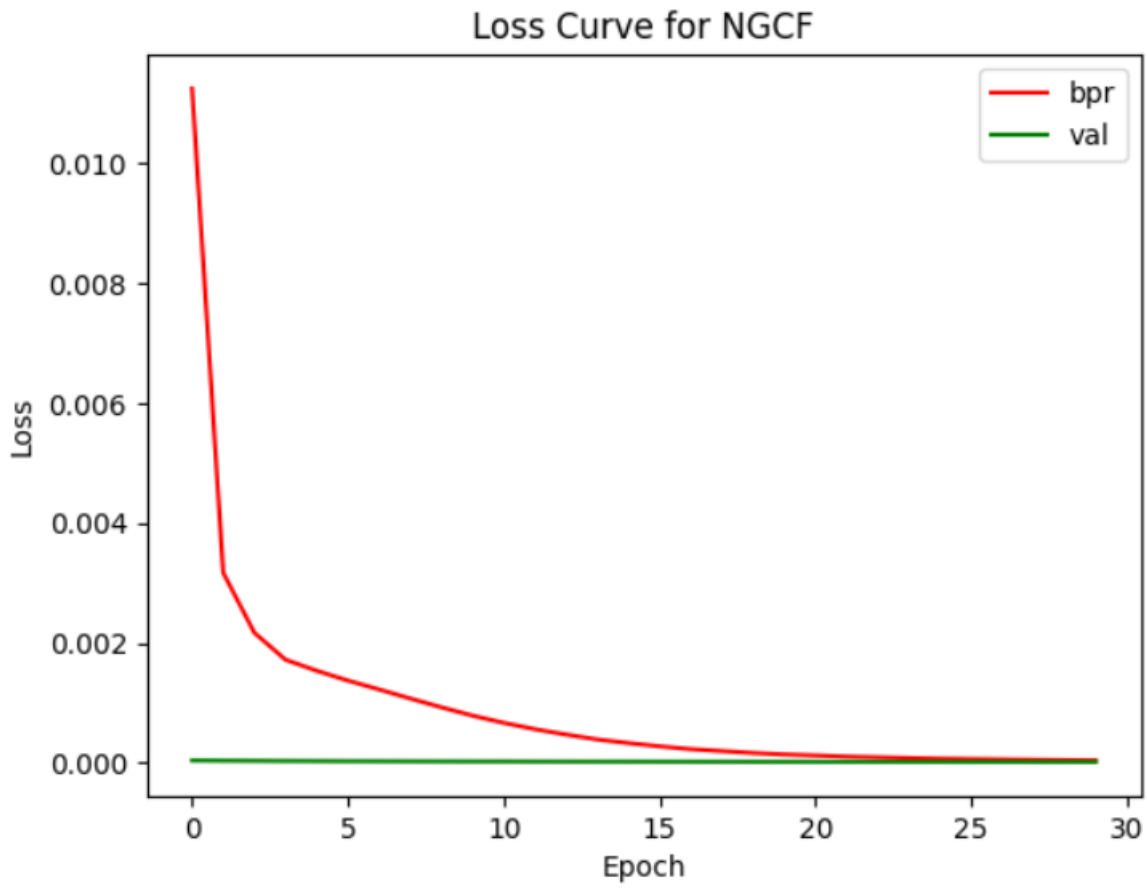

Fig. 4 Loss curve for Neural Graph Collaborative Filtering.

- Price
- Brand
- Image Links (if any)
- Also Viewed Products
- Also Bought Products

In this recommendation analysis, we are using the "*also bought*" products as the target variable, trying to predict them using various approaches.

3.2 Software and Hardware Setup

The experiments and analysis for this project have been conducted either on local computer systems and/or Google Colab notebooks for faster processing. Python was the programming language used. Some of the prominent libraries uses for model building and data processing include PyTorch, Scikit-learn, PyTorch Geometric, DeepGraphLibrary (dgl), Pandas.

We have used Indiana University Big Red 200 and Carbonate Supercomputers for running our experiments.

3.3 Hyperparameters for Models

3.3.1 GraphSAGE

For GraphSAGE, the hyperparameters that were tuned include the following:

- Number of Layers in the model
- Learning Rate
- Hidden Layers Dimension

3.3.2 NGCF

For NGCF, the hyperparameters that were tuned include the following:

- Number of Layers in the model
- Learning Rate
- Message Dropout
- Node Dropout
- Negative Sampling

4 Results

4.1 GraphSAGE

We run cross-validation with HitRate and Area Under Curve (AUC) as parameters to find the best hyperparameters for the model. We test 2 layer, 4 layer and 8 layer deep models with learning rates of 0.001, 0.01, 0.02 and hidden dimensions of 8, 32, 64, 128, 256 and 512. We find that a 4 layer deep model with a learning rate 0.001 and hidden dimension of 64 performs the best. We then train a fresh model with this parameters and check for further details. We notice that the performance of the model degrades after the hidden dimension is increased after 64, falling dramatically for 512 dimensions. Similarly, there is lower performance for a learning rate of 0.02, implying it's higher than required. The 8 layer deep model performs slightly worse than the finalized 4 layer model while the 2 layer model performs the worst.

Table 1 Table 2: Hyperparameter tuning for GraphSAGE

Num. Layers	Learning Rate	Hidden Dim.	HitRate
4	0.001	64	**0.846**
8	0.02	64	0.76
2	0.02	64	0.69
4	0.01	64	0.65

As seen in Fig 1, the training loss continues to fall until the end while the test loss starts plateauing after around 40 epochs. It stays above training loss but still gives a good performance which is a a HitRate of 0.846. This is much better than a traditional collaborative filtering algorithm although slightly lower than the NGCF model mentioned below.

4.2 NGCF

We tested out model for a range of parameters mentioned above. For the above hyperparameters, that we mentioned, the learning rate of 0.001 gives the optimal hit rate of 0.8709.The number of layers didn't affect the hit rate to a greater extent, since we tried the model only on 2,3 and 4 layers due to the computational limitations, the model with depth of 3 was slightly better.While tuning the dropout strategy, we tested effects of both dropouts - the node dropout as well and the range of both the values was between 0.1 and 0.8 (both included).It was observed that the best message dropout value of 0.1 gives the best performance.Increasing the value of message dropout rate causes the degradation in the model performance, causing the hit rate to drop by 0.015. This might be due to the fact that increasing the message dropout causes the model to forego complex information.While on the other hand, increasing the node dropout rate slightly improves the hit rate by 0.030, this might have happened due to the fact that dropping of messages from particular user or item, makes the representations robust against particular influencing edges.

The negative sampling ratio determines the number of negative samples generated for each positive sample.In our case that ratio was 1:7(positive:negative samples), which gave optimal hit rate.Higher ratio than that mentioned above resulting in the oscillation of hit rate value.

Table 2 Table 1: Hyperparameter tuning for NGCF - Layers

Number of Layers	**Hit Rate**
2	0.8563
3	0.8709
4	0.8580

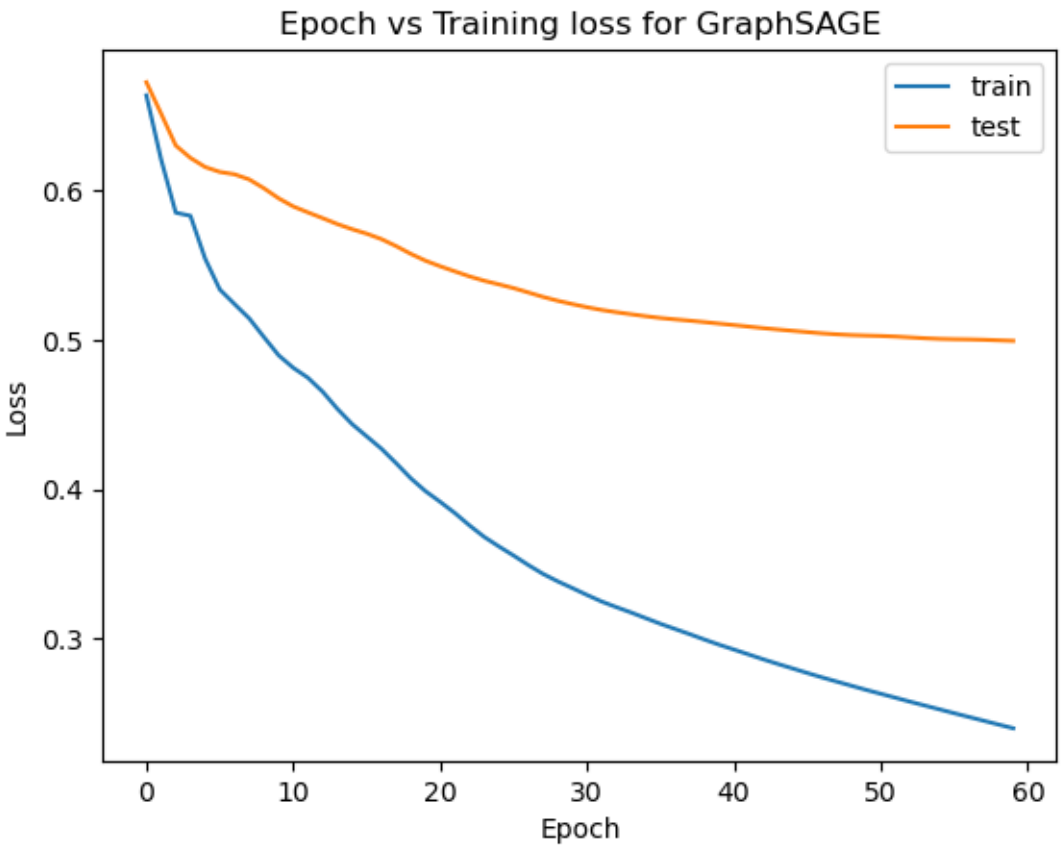

Fig. 5 Loss vs Epoch for GraphSAGE

Table 3 Table 2: Hyperparameter tuning for NGCF - Negative Sampling Ratio

Sampling Ratio	Hit Rate
5	0.8498
6	0.8654
7	0.8709
8	0.8685
9	0.8600

5 Conclusions

5.1 Key Takeaways

We observe that graph neural network based models outperform traditional collaborative filtering based algorithms comprehensively. However, the traditional collaborative filtering can be used as a baseline for other models to surpass. Neural Graph Collaborative Filtering performs the best while GraphSAGE performs slighlty worser than it. Both the models achieve a hit rate of over 80 percent, thereby making them both strong candidates for performance. The choice of model selection depends on the available data and it's format. GraphSAGE can be preferred when there are many informative features that can be harnessed, whereas NGCF can be used when we have large graphs with sparsity. Hyperparameter tuning and training strategies can be used to further improve the performance of these models.

5.2 Limitations and Future Work

The current analysis only deals with products belonging to one category, owing to computational limitations. However, the same analysis can be extended to be done over the entire dataset covering millions of nodes. This will improve the performance and generalizability of the techniques even more. One of the major drawback with the traditional collaborative filtering is that it cannot be scaled for millions of products [5] and performance reduces drastically drastically. Also traditional models cannot incorporate all the

Table 4 Model Performance Comparison

Model	HitRate
Traditional CF	0.612
GraphSAGE	0.846
NGCF	**0.871**

user/item features while making recommendations. For graph neural networks, the GraphSAGE model can be leveraged further more by considering further features like sentiment analysis and image analysis from the dataset. Some other graph embedding algorithms like LightGCN, Graph Attention Networks (GAT) can also be used and their performance benchmarked.

6 Team Member Contributions

- **Soham Bhagwat**: Overall research and information gathering, Traditional Collaborative filtering techniques, Neural Graph Collaborative Filtering (NGCF) primary author, Project Report preparation.
- **Srikar Kashyap Pulipaka**: Overall research and information gathering, Traditional Collaborative filtering techniques, GraphSAGE primary auhtor, Project Report preparation.

References

[1] Rajamanickam, S.: Graph Neural Network (GNN) architectures for recommendation systems. Towards Data Science (2021). https://towardsdatascience.com/graph-neural-network-gnn-architectures-for-recommendation-systems-7b9dd0de0856

[2] Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. Advances in Neural Information Processing Systems, 1024–1034 (2017)

[3] Wang, X., He, X., Wang, M., Feng, F., Chua, T.: Neural graph collaborative filtering. CoRR **abs/1905.08108** (2019) 1905.08108

[4] Ni, J., Li, J., McAuley, J.: Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 188–197. Association for Computational Linguistics, Hong Kong, China (2019). https://doi.org/10.18653/v1/D19-1018 . https://aclanthology.org/D19-1018

[5] Linden, G., Smith, B., York, J.: Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Distributed Syst. Online **4** (2003)